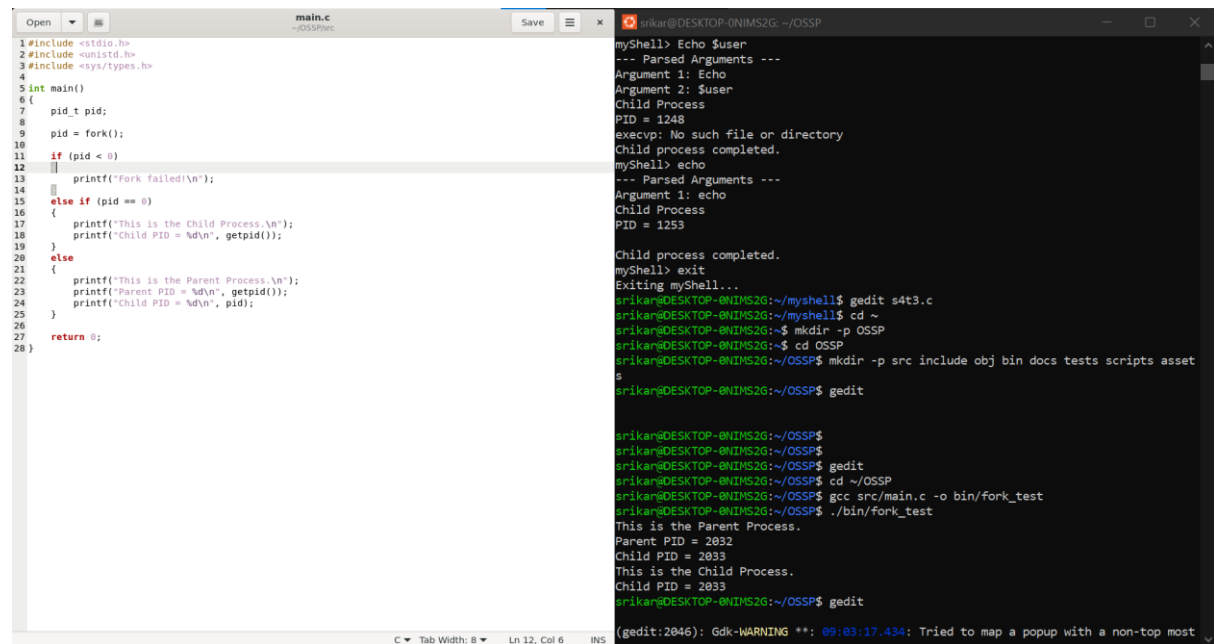


Skill-2:-

1.



The image shows a C program in a text editor and its execution in a terminal. The C program, named `main.c`, uses `fork()` to create a child process. It prints the parent and child PIDs and handles errors. The terminal shows the program being compiled and run, demonstrating the creation of a child process and the execution of a shell.

```
1#include <stdio.h>
2#include <unistd.h>
3#include <sys/types.h>
4
5int main()
6{
7    pid_t pid;
8
9    pid = fork();
10
11    if (pid < 0)
12    {
13        printf("Fork failed!\n");
14    }
15    else if (pid == 0)
16    {
17        printf("This is the Child Process.\n");
18        printf("Child PID = %d\n", getpid());
19    }
20    else
21    {
22        printf("This is the Parent Process.\n");
23        printf("Parent PID = %d\n", getpid());
24        printf("Child PID = %d\n", pid);
25    }
26
27    return 0;
28}
```

```
myShell> Echo $user
--- Parsed Arguments ---
Argument 1: Echo
Argument 2: $user
Child Process
PID = 1248
execvp: No such file or directory
Child process completed.
myShell> echo
--- Parsed Arguments ---
Argument 1: echo
Child Process
PID = 1253
Child process completed.
myShell> exit
Exiting myShell...
srikar@DESKTOP-0NIMS2G: ~/myshell$ gedit s4t3.c
srikar@DESKTOP-0NIMS2G: ~/myshell$ cd ~
srikar@DESKTOP-0NIMS2G: ~$ mkdir -p OSSP
srikar@DESKTOP-0NIMS2G: ~$ cd OSSP
srikar@DESKTOP-0NIMS2G: ~/OSSP$ mkdir -p src include obj bin docs tests scripts asset
s
srikar@DESKTOP-0NIMS2G: ~/OSSP$ gedit
srikar@DESKTOP-0NIMS2G: ~/OSSP$
srikar@DESKTOP-0NIMS2G: ~/OSSP$
srikar@DESKTOP-0NIMS2G: ~/OSSP$ gedit
srikar@DESKTOP-0NIMS2G: ~/OSSP$ cd ~/OSSP
srikar@DESKTOP-0NIMS2G: ~/OSSP$ gcc src/main.c -o bin/fork_test
srikar@DESKTOP-0NIMS2G: ~/OSSP$ ./bin/fork_test
This is the Parent Process.
Parent PID = 2032
Child PID = 2033
This is the Child Process.
Child PID = 2033
srikar@DESKTOP-0NIMS2G: ~/OSSP$ gedit
(gedit:2046): Gdk-WARNING **: 09:03:17.434: Tried to map a popup with a non-top most
```

2.

Task 1:

1. fork documentation

```
srikar@DESKTOP-0NIMS2G: ~/OSSP
fork(2)                                System Calls Manual                                fork(2)
NAME
    fork - create a child process

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <unistd.h>

    pid_t fork(void);

DESCRIPTION
    fork() creates a new process by duplicating the calling process. The new
    process is referred to as the child process. The calling process is re-
    ferred to as the parent process.

    The child process and the parent process run in separate memory spaces.
    At the time of fork() both memory spaces have the same content. Memory
    writes, file mappings (mmap(2)), and unmappings (munmap(2)) performed by
    one of the processes do not affect the other.

    The child process is an exact duplicate of the parent process except for
    the following points:

    • The child has its own unique process ID, and this PID does not match
      the ID of any existing process group (setpgid(2)) or session.

    • The child's parent process ID is the same as the parent's process ID.

    • The child does not inherit its parent's memory locks (mlock(2), mlock-
      all(2)).

    • Process resource utilizations (getrusage(2)) and CPU time counters
      (times(2)) are reset to zero in the child.

    • The child's set of pending signals is initially empty (sigpending(2)).

Manual page fork(2) line 1 (press h for help or q to quit)_
```

2. exec documentation

```
srikar@DESKTOP-0NIMS2G: ~/OSSP
exec(3) Library Functions Manual exec(3)
NAME
    execl, execlp, execl, execv, execvp, execvpe - execute a file
LIBRARY
    Standard C library (libc, -lc)
SYNOPSIS
    #include <unistd.h>

    extern char **environ;

    int execl(const char *pathname, const char *arg, ...
                /*, (char *) NULL */);
    int execlp(const char *file, const char *arg, ...
                /*, (char *) NULL */);
    int execl(const char *pathname, const char *arg, ...
                /*, (char *) NULL, char *const envp[] */);
    int execv(const char *pathname, char *const argv[]);
    int execvp(const char *file, char *const argv[]);
    int execvpe(const char *file, char *const argv[], char *const envp[]);

    Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

    execvpe():
        _GNU_SOURCE

DESCRIPTION
    The exec() family of functions replaces the current process image with a
    new process image. The functions described in this manual page are lay-
    ered on top of execve(2). (See the manual page for execve(2) for further
    details about the replacement of the current process image.)

    The initial argument for these functions is the name of a file that is to
    be executed.

    The functions can be grouped based on the letters following the "exec"
    prefix.

Manual page exec(3) line 1 (press h for help or q to quit)
```

3. Process tree

```

srikar@DESKTOP-0NIMS2G:~/OSSP$ man 3 exec
srikar@DESKTOP-0NIMS2G:~/OSSP$ pstree -p
systemd(1)─agetty(206)
          └─cron(172)
          └─dbus-daemon(173)
          └─init-systemd(Ub(2)
            └─SessionLeader(302)─Relay(304)(303)─bash(304)─+++
              └─init(7)─{init}(8)
              └─login(305)─bash(377)
              └─{init-systemd(Ub)}(9)
          └─rsyslogd(204)─{rsyslogd}(226)
                        └─{rsyslogd}(227)
                        └─{rsyslogd}(228)
          └─systemd(351)─(sd-pam)(352)
                        └─at-spi-bus-laun(666)─dbus-daemon(673)
                                              └─{at-spi-bus-laun}(667)
                                              └─{at-spi-bus-laun}(668)
                                              └─{at-spi-bus-laun}(669)
                                              └─{at-spi-bus-laun}(671)
                        └─at-spi2-registr(701)─{at-spi2-registr}(702)
                                              └─{at-spi2-registr}(703)
                                              └─{at-spi2-registr}(704)
                        └─dbus-daemon(662)
                        └─dconf-service(738)─{dconf-service}(739)
                                              └─{dconf-service}(740)
                                              └─{dconf-service}(741)
                        └─xdg-desktop-por(674)─{xdg-desktop-por}(675)
                                              └─{xdg-desktop-por}(676)
                                              └─{xdg-desktop-por}(677)
                                              └─{xdg-desktop-por}(706)
                        └─xdg-desktop-por(694)─{xdg-desktop-por}(695)
                                              └─{xdg-desktop-por}(696)
                                              └─{xdg-desktop-por}(698)
                                              └─{xdg-desktop-por}(699)
                        └─xdg-document-po(679)─fusermount3(690)
                                              └─{xdg-document-po}(680)
                                              └─{xdg-document-po}(681)
                                              └─{xdg-document-po}(682)
                                              └─{xdg-document-po}(689)
                                              └─{xdg-document-po}(691)
                                              └─{xdg-document-po}(692)
                        └─xdg-permission-(683)─{xdg-permission-}(684)
                                              └─{xdg-permission-}(685)
                                              └─{xdg-permission-}(687)
          └─systemd-journal(1824)
          └─systemd-logind(183)
          └─systemd-resolve(132)
          └─systemd-timesyn(136)─{systemd-timesyn}(170)
          └─systemd-udevd(101)
          └─unattended-upgr(216)─{unattended-upgr}(263)
          └─wsl-pro-service(186)─{wsl-pro-service}(218)
                                └─{wsl-pro-service}(219)
                                └─{wsl-pro-service}(220)
                                └─{wsl-pro-service}(221)
                                └─{wsl-pro-service}(222)
                                └─{wsl-pro-service}(223)
                                └─{wsl-pro-service}(224)
srikar@DESKTOP-0NIMS2G:~/OSSP$

```

4. Process information

```
srikar@DESKTOP-0NIMS2G: ~/OSSP
├─dbus-daemon(662)
├─dconf-service(738)─┬─{dconf-service}(739)
│                   └─{dconf-service}(740)
│                   └─{dconf-service}(741)
├─xdg-desktop-por(674)─┬─{xdg-desktop-por}(675)
│                   └─{xdg-desktop-por}(676)
│                   └─{xdg-desktop-por}(677)
│                   └─{xdg-desktop-por}(706)
├─xdg-desktop-por(694)─┬─{xdg-desktop-por}(695)
│                   └─{xdg-desktop-por}(696)
│                   └─{xdg-desktop-por}(698)
│                   └─{xdg-desktop-por}(699)
├─xdg-document-po(679)─┬─fusermount3(690)
│                   └─{xdg-document-po}(680)
│                   └─{xdg-document-po}(681)
│                   └─{xdg-document-po}(682)
│                   └─{xdg-document-po}(689)
│                   └─{xdg-document-po}(691)
│                   └─{xdg-document-po}(692)
├─xdg-permission-(683)─┬─{xdg-permission-}(684)
│                   └─{xdg-permission-}(685)
│                   └─{xdg-permission-}(687)
├─systemd-journal(1824)
├─systemd-logind(183)
├─systemd-resolve(132)
├─systemd-timesyn(136)─┬─{systemd-timesyn}(170)
├─systemd-udevd(101)
├─unattended-upgr(216)─┬─{unattended-upgr}(263)
├─wsl-pro-service(186)─┬─{wsl-pro-service}(218)
│                   └─{wsl-pro-service}(219)
│                   └─{wsl-pro-service}(220)
│                   └─{wsl-pro-service}(221)
│                   └─{wsl-pro-service}(222)
│                   └─{wsl-pro-service}(223)
│                   └─{wsl-pro-service}(224)
srikar@DESKTOP-0NIMS2G:~/OSSP$ ps -f
UID      PID    PPID  C  STIME TTY          TIME CMD
srikar    304     303  0  08:05 pts/0        00:00:00 -bash
srikar   2310     304  50  09:11 pts/0        00:00:00 ps -f
srikar@DESKTOP-0NIMS2G:~/OSSP$
```

5. System call tracing

```
srikar@DESKTOP-0NIMS2G: ~/OSSP
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_COLLATE", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_COLLATE", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=1406, ...}) = 0
mmap(NULL, 1406, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c3035439000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_TIME", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such fil
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_TIME", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=3360, ...}) = 0
mmap(NULL, 3360, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c3035438000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_NUMERIC", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_NUMERIC", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=50, ...}) = 0
mmap(NULL, 50, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c3035437000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_CTYPE", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such fi
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_CTYPE", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=360460, ...}) = 0
mmap(NULL, 360460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c30351a7000
close(3) = 0
ioctl(1, TCGETS, {c_iflag=ICRNL|IXON, c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|OPOST|ONLCR, c_cflag=B384
CHOCTL|ECHOKE, ...}) = 0
ioctl(1, TIOCGWINSZ, {ws_row=81, ws_col=155, ws_xpixel=0, ws_ypixel=0}) = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
fstat(3, {st_mode=S_IFDIR|0755, st_size=4096, ...}) = 0
getdents64(3, 0x5d4af3bdd8a0 /* 10 entries */, 32768) = 272
getdents64(3, 0x5d4af3bdd8a0 /* 0 entries */, 32768) = 0
close(3) = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
write(1, "assets\tbin docs include obj "..., 52assets      bin docs include obj scripts
) = 52
close(1) = 0
close(2) = 0
exit_group(0) = ?
+++ exited with 0 +++
srikar@DESKTOP-0NIMS2G:~/OSSP$
```

TASK 2:-

```
main1.c
~OSSP/src

21 {
22     if (strcmp(input, "") == 0)
23     {
24         printf("Empty command.\n");
25         continue;
26     }
27
28     /* Exit command */
29     if (strcmp(input, "exit") == 0)
30     {
31         printf("Exiting shell...\n");
32         break;
33     }
34
35     /* Create child process */
36     pid_t pid = fork();
37
38     if (pid < 0)
39     {
40         perror("fork");
41         continue;
42     }
43
44     /* Child process */
45     if (pid == 0)
46     {
47         printf("Child Process\n");
48         printf("Child PID = %d\n", getpid());
49
50         /*
51          * Execute the user's command
52          */
53         execl("/bin/sh", "sh", "-c", input, NULL);
54
55         /* execl returns only if an error occurs */
56         perror("execl");
57         exit(EXIT_FAILURE);
58     }
59
60     /* Parent process */
61     else
62     {
63         printf("Parent PID = %d\n", getpid());
64         printf("Child PID = %d\n", pid);
65
66         /* Wait for child */
67         waitpid(pid, NULL, 0);
68
69         printf("Command execution completed.\n");
70     }
71
72     return 0;
73 }
```

```
srirak@DESKTOP-0NIMS2G: ~/OSSP
mmap(NULL, 3360, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c3035438000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_NUMERIC", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_NUMERIC", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=50, ...}) = 0
mmap(NULL, 50, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c3035437000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_CTYPE", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_CTYPE", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=360460, ...}) = 0
mmap(NULL, 360460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c30351a7000
close(3) = 0
ioctl(1, TCGETS, {c_iflag=ICRNL|IXON, c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|OPOST|ONLCR, c_cflag=B38400, c_lflag=0, ...}) = 0
ioctl(1, TIOCGWINSZ, {ws_row=81, ws_col=155, ws_xpixel=0, ws_ypixel=0}) = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
fstat(3, {st_mode=S_IFDIR|0755, st_size=4096, ...}) = 0
getdents64(3, 0x5d4af3bdd8a0 /* 10 entries */, 32768) = 272
getdents64(3, 0x5d4af3bdd8a0 /* 0 entries */, 32768) = 0
close(3) = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
write(1, "assets/tbin docs include obj "..., 52assets      bin docs include obj scripts
) = 52
close(1) = 0
close(2) = 0
exit_group(0) = ?
+++ exited with 0 +++
srirak@DESKTOP-0NIMS2G:~/OSSP$ cd ~/OSSP
srirak@DESKTOP-0NIMS2G:~/OSSP$ gedit src/main1.c
srirak@DESKTOP-0NIMS2G:~/OSSP$ gcc -o bin/my_shell
gcc: error: unrecognized command-line option '-wextra'; did you mean '-Wall'?
srirak@DESKTOP-0NIMS2G:~/OSSP$ gcc -Wall -wextra src/main.c -o bin/my_shell
srirak@DESKTOP-0NIMS2G:~/OSSP$ ./bin/my_shell
This is the Parent Process.
Parent PID = 2615
Child PID = 2616
This is the Child Process.
Child PID = 2616
srirak@DESKTOP-0NIMS2G:~/OSSP$ gedit src/main1.c
```